PRÜFUNGSSTUDIENARBEIT

Technische Hochschule Deggendorf

Fakultät Angewandte Informatik

Bachelor Künstliche Intelligenz

Fach: C in der automobilen Softwareentwicklung

Thema: Implementierung und Analyse numerischer Integrationsverfahren in C

Autor:

Peter Okruhlica

Matrikelnummer:

22300246

Prüfer:

Prof. Thomas Limbrunner

Abgabedatum: Deggendorf, 15.01.2026

Ich versichere, dass ich die beiliegende Arbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe. Die den benutzten Quellen wörtlich oder inhaltlich entnommenen Inhalte sind als solche kenntlich gemacht. Die vorliegende Studienarbeit wurde bei keiner anderen Prüfung vorgelegt. Ich bin damit einverstanden, dass die Ergebnisse dieser Prüfungsstudienarbeit honorarfrei für Forschungs- und Lehrzwecke genutzt werden können.

Plattling, 15.01.2026

……..

Ort, Datum

……..

Unterschrift

Inhaltsverzeichnis

1 Softwarearchitektur 1

1.1 Die gewählte Softwarearchitektur 1

2 Theoretische Grundlagen 2

2.1 Die Rechteckmethode 2

2.2 Die Trapezmethode 3

2.3 Die Simpsonmethode 4

3 Tests und Analyse der Ergebnisse 5

3.1 Setup der Tests 5

3.2 Analyse der Ergebnisse 5

3.2.1 Ergebnisse der Polynomfunktion 6

3.2.2 Ergebnisse der trigonometrischen Funktion 6

3.3 Limitation der Rechteckmethode 6

3.4 Limitation der Simpsonmethode 6

3.5 Fazit 7

4 Glossar der Abkürzungen 8

5 Appendix 9

5.1 Berechnung des Integrals der Polynomfunktion 9

5.2 Berechnung des Integrals der trigonometrischen Funktion 9

6 Nutzung von KI 10

6.1 Prompt 1 10

6.2 Prompt 2 10

7 Literaturverzeichnis 10

1 Softwarearchitektur

Die Implementierung der numerischen Integrationsverfahren sieht mehrere technische Requirements vor. Für die Wahl der passenden Softwarearchitektur spielen folgende Requirements eine wichtige Rolle: Reusability des Moduls für Integration und ein Vergleich der Integrationsverfahren im Hinblick auf ihre Genauigkeit. Im folgenden Abschnitt wird die gewählte Architektur vorgestellt und erläutert, wie die identifizierten Requirements innerhalb der Architektur umgesetzt werden.

1.1 Die gewählte Softwarearchitektur

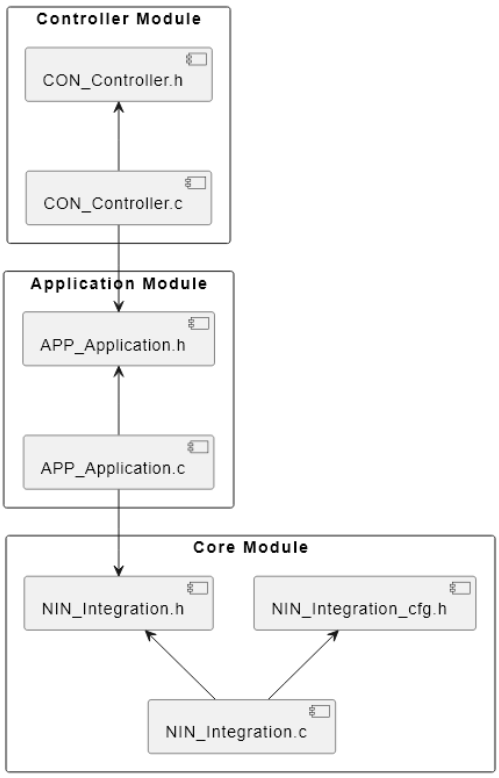

Bild 1 – Architektur der Anwendung

Die Architektur wird in 3 Modulebenen unterteilt:

- **Controller Modul,** steuert den Programmfluss.
- **Application Modul,** kapselt die Anwendungslogik und testet die Verfahren auf Genauigkeit.
- **Core Modul,** implementiert die Integrationsverfahren.

Diese Architektur wurde so gewählt, um eine klare Trennung zwischen Steuerung (CON), Anwendungslogik (APP) und mathematischem Kern (NIN) zu gewährleisten. Die jeweiligen Pfeile zeigen die *#include* Statements. Auf spezifische *includes* wie z.B.: <stdio.h> oder „global.h“ wird hier verzichtet. Dem User ist es möglich, zwischen **Berechnen-Modi** ODER **Testen-Modi** zu wählen. Im Berechnen-Modus wird die gewählte Funktion numerisch integriert und die approximierten Ergebnisse der drei Integrationsfunktionen aus *NIN_Integration.c* werden in der Konsolenanwendung angezeigt. Im Testen-Modus werden die vordefinierten Tests ausgeführt und die Ergebnisse angezeigt. In beiden Modi ist es dem User möglich zwischen trigonometrischer oder polynomischer Funktion zu wählen. Auf weitere benutzerdefinierte Einstellungen zur Laufzeit wird verzichtet. Der Vorteil dieses Ansatzes liegt in der Verwendung der Funktionspointer, um beliebige Funktionen und deren Integrale zu berechnen. Dadurch ist das Integrationsmodul vollständig entkoppelt. Die Ergebnisse werden ebenfalls über (Struktur-)Pointer von APP-Modul an CON-Modul weitergegeben. Die jeweiligen Abhängigkeiten können auch anhand der Design-Struktur-Matrix überprüft werden.

	CON.c	CON.h	APP.c	APP.h	NIN.c	NIN.h
CON.c		X		X		
CON.h						
APP.c				X		X
APP.h						
NIN.c						X
NIN.h						

Tabelle 1 – Design-Struktur-Matrix

Die Leserichtung ist „Zeile bindet Spalte ein“ z.B.: APP.c bindet die APP.h und NIN.h Headerdateien ein. Auch hier spiegeln sich die umgesetzten Konzepte wider.

2 Theoretische Grundlagen

Der Kern der Aufgabe liegt in der Implementierung von drei numerischen Integrationsverfahren, sodass sie für die Anwendung auf einem Embedded Controller optimiert sind. Zu implementieren sind: die Rechteckmethode, die Trapezmethode und die Simpsonmethode. Im Folgenden werden die Methoden, sowohl mathematisch als auch algorithmisch, näher vorgestellt. Der Einfachheit halber werden noch die Variablen erklärt, die bei allen Methoden gleich sind. Dabei bezeichnen a und b die untere bzw. obere Integrationsgrenze des Intervalls $[\mathrm{a}, \mathrm{b}]$.[1] Der Parameter n steht für die Anzahl der äquidistanten Unterteilungen.[1] Daraus resultiert die konstante Schrittweite h, die sich als $h = \frac{b-a}{n}$ berechnet.[1]

Aus mathematischer Sicht muss bei allen 3 Funktionen auf 2 Parameter aufgepasst werden, nämlich auf den Parameter n und das Intervall $[\mathrm{a}, \mathrm{b}]$. Dieser darf nicht $\mathrm{n} <= 0$ und die Intervalldifferenz darf nicht $<= 0$ sein. Ein Intervall kann zwar nicht (mathematisch) negativ werden, so kann aber z.B.: eine fehlerhafte Eingabe von $[0, 0]$ oder $[-2, -5]$ zu ungültigen Ergebnissen führen. Dies wird entsprechend der Vorgaben bei der Implementierung beachtet und behandelt. Die maximale Anzahl der äquidistanten Unterteilungen wird auf $\mathrm{n} = 1.000$ gesetzt. Das NIN-Modul wird als reine mathematische Library implementiert, d.h. es werden nur die Funktionen unabhängig voneinander implementiert. Das NIN-Modul soll die Aufgabe eines Low-Level-Moduls erfüllen und somit leicht in andere Projekte integrierbar sein.

2.1 Die Rechteckmethode

Die Rechteckmethode ist die einfachste von den drei zu implementieren. Sie basiert auf der Idee des Riemannschen Integrals, die Fläche unter der Kurve einer Funktion durch Flächen der Rechtecke zu bestimmen [1, S. 3]. Das bestimmte Integral einer gegebenen Funktion $f(x)$ lässt sich durch folgende Formel annähern:

[1] 6.1 Prompt 1

$$\int_a^b f(x)dx \approx h * \sum_{i=0}^{n-1} f\,(a + i * h)$$

Formel 1 – Rechteckmethode [1, S. 4]

Der Parameter n bestimmt die Granularität, wie viele Rechtecke unter die Kurve platziert werden sollen. Je mehr Rechtecke unter der Kurve, desto präziser wird die Annäherung des Integrals. Die Rechteckmethode implementiert die Methode der Untersumme, d.h. die Rechtecke bleiben stets unter der Kurve (vgl. Bild 2). Umgesetzt in C wird es wie folgt: Alle möglichen Variablen oder Koeffizienten werden nach der Prüfung der übergebenen Parameter berechnet bzw. festgelegt, damit diese bereit für die nächsten Berechnungen zur Verfügung stehen. Anschließend folgt ein for-loop für den Summationsteil. Zum Schluss wird das Ergebnis der Summation mit dem Koeffizienten h multipliziert und das Ergebnis in einen Pointer gespeichert.

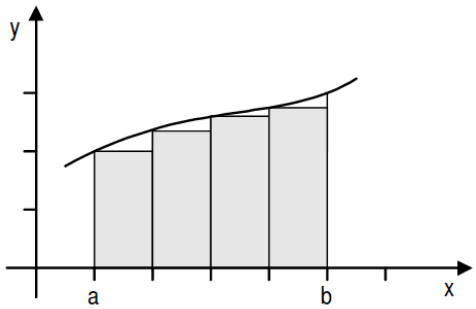

Bild 2 – Integration Untersumme [1, S. 5]

2.2 Die Trapezmethode

Die Trapezmethode ist sehr ähnlich zu der Rechteckmethode und unterscheidet sich nur in der Art, wie das Integral bestimmt wird, enthält aber dieselben Parameter wie die Rechteckmethode. Statt Rechtecke unter die Kurve zu platzieren, werden Trapeze platziert. Diese Methode deckt viel besser die Fläche unter der Kurve ab und kann präziseres Integral liefern [1, S. 5]. Das bestimmte Integral einer gegebenen Funktion $f(x)$ lässt sich durch folgende Formel annähern:

$$\int_a^b f(x)dx \approx \frac{h}{2} * \{f(a) + \left(\sum_{i=1}^{n-1} 2f(a + i * h)\right) + f(b)\}$$

Formel 2 – Trapezmethode [1, S. 6]

Auch hier gilt, je mehr Trapeze unter der Kurve, also je höheres n, desto präziseres Ergebnis liefert die Funktion. Umgesetzt in C wird es wie folgt: Nach der Prüfung aller Parameter

werden die Koeffizienten und Funktionswerte der Ränder gebildet. Dann wird die Summation mit einem for-loop realisiert. Am Ende wird eine gemeinsame Summe, aus dem for-loop und der Funktionswerte der Ränder gebildet, die mit dem Termkoeffizienten multipliziert wird. Das fertige Ergebnis wird in einen Pointer gespeichert.

2.3 Die Simpsonmethode

Die Simpsonmethode unterscheidet sich von den vorherigen 2 Verfahren. Sie beruht nicht mehr auf der Idee des Riemannschen Integrals, sondern auf der Idee der Keplerschen Fassregel [1, S. 7]. Dabei werden keine klassischen geometrischen Strukturen unter die Kurve platziert, sondern wird die Fläche unter der Kurve durch Parabelbögen angenähert (vgl. Bild 3) und somit kann ein präzises Ergebnis bereits durch kleine Anzahl von n erreicht werden [1, S. 7].

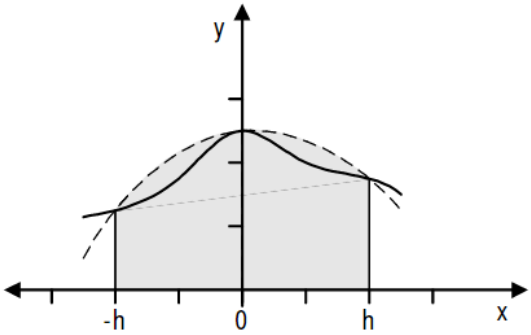

Bild 3 – Keplersche Fassregel [1, S. 7]

Das bestimmte Integral einer gegebenen Funktion $f(x)$ lässt sich durch folgende Formel annähern:

$$\int_a^b f(x)dx \approx \frac{h}{3} * \left\{ f(a) + \left(4 \sum_{i=1}^{\frac{n}{2}} f(a + (2i-1) * h) + 2 \sum_{i=1}^{\frac{n}{2}-1} f(a + (2i) * h) \right) + f(b) \right\}$$

Formel 3 – Simpsonmethode [1, S. 9]

Allerdings hat diese Formel eine Einschränkung. Bei ungeradem n kann diese Funktion ungenaue Ergebnisse liefern. Dies wird näher in Kapitel **3.4 Limitation der Simpsonmethode** erläutert. Umgesetzt in C wird es wie folgt: Alle möglichen Variablen, Koeffizienten sowie die Funktionswerte der Ränder werden nach der Prüfung der übergebenen Parameter berechnet bzw. festgelegt. Anschließend folgen zwei separate for-loops für die Summationsteile, um die ungeraden und geraden Stützstellen getrennt zu verarbeiten. Zum Schluss werden die Teilsummen addiert, mit dem Koeffizienten multipliziert und das Ergebnis in einen Pointer gespeichert.

3 Tests und Analyse der Ergebnisse

3.1 Setup der Tests

Für das Testen der Genauigkeit wurden folgende Funktionen gewählt:

1. Polynomische Funktion 4. Grades: $p(x) = 4x^4 + 7x^3 - 4x + 14$
2. Trigonometrische Funktion: $t(x) = 18 * \sin(x)$

Die Polynomfunktion dient der Evaluierung der numerischen Verfahren bei starkem Anstieg und großen Flächeninhalten.[2] Im Gegensatz dazu zielt der Einsatz der trigonometrischen Funktion darauf ab, die Robustheit der Algorithmen bei Vorzeichenwechseln sowie den Umgang mit negativen Integralbeiträgen zu untersuchen.[2]

Die Genauigkeit wird für 3 unterschiedliche n-Werte untersucht, nämlich $\mathrm{n} = \{10, 50, 100\}$. Das Intervall wird auf $[0, 20]$ festgelegt. Die tatsächlichen Ergebnisse für die beiden Funktionen sind ebenfalls in CON_Controller.c hartcodiert. Die Herleitung dieser Ergebnisse können Sie dem **5 Appendix** entnehmen. All diese Werte sind zur Laufzeit des Programmes nicht änderbar.

3.2 Analyse der Ergebnisse

Die Tests und deren Ergebnisse können in der Konsolenanwendung mit der Wahl der Nr.3 oder Nr. 4 reproduziert werden. Die Tests liefern folgende Ergebnisse:

```
--- Accuracy Results (Polynomial) ---

N = 10:
Rectangle: 77.091%
Trapezoid: 101.600%
Simpson:   100.006%

N = 50:
Rectangle: 95.162%
Trapezoid: 100.064%
Simpson:   100.000%

N = 100:
Rectangle: 97.565%
Trapezoid: 100.016%
Simpson:   100.000%
```

Bild 4 – Ergebnisse Polynomfunktion

```
--- Accuracy Results (Trigonometric) ---

N = 10:
Rectangle: -90.022%
Trapezoid: 64.206%
Simpson:   116.118%

N = 50:
Rectangle: 67.813%
Trapezoid: 98.659%
Simpson:   100.010%

N = 100:
Rectangle: 84.239%
Trapezoid: 99.662%
Simpson:   99.996%
```

Bild 5 – Ergebnisse trigonometrische Funktion

Für die Auswertung der Ergebnisse nehmen wir an, dass in den Akzeptanzkriterien eine maximal erlaubte Abweichung von ± 2% festgelegt wurde.

[2] 6.2 Prompt 2

3.2.1 Ergebnisse der Polynomfunktion

Die Rechteckmethode schneidet bei der Polynomfunktion am schlechtesten ab. Für einen kleinen Wert $n=10$ liefert diese Methode ein approximiertes Ergebnis mit einer 22,9% Abweichung von tatsächlichem Wert. Dies stellt eine erhebliche Abweichung dar und kann nicht als zuverlässiges Ergebnis betrachtet werden. Die Rechteckmethode wird mit größerem n immer präziser, allerdings erreicht sie in keinem der 3 Tests die maximal erlaubte Abweichung. Die Trapezmethode überschätzt zwar das tatsächliche Ergebnis um 1,6% bei $n=10$, kann aber akzeptiert werden. Die Simpsonmethode liefert nahezu perfekte Ergebnisse bei allen 3 Tests. Die Abweichung bei $n=10$ beträgt nur 0,006% und bei höheren Werten von n stimmt das approximierte Ergebnis mit dem Tatsächlichen überein.

3.2.2 Ergebnisse der trigonometrischen Funktion

Genauso hier schneidet die Rechteckmethode am schlechtesten ab. Für den Wert $n=10$ liefert diese Funktion ein Ergebnis mit einer Abweichung von 190,022%. Die Funktion liefert ein Ergebnis mit einem falschen Vorzeichen und stellt massive Abweichung dar. Die anderen beiden Methoden liefern hier auch unzufriedenstellende Ergebnisse mit Abweichungen von 35,794% bei der Trapez- und 16,118% bei der Simpsonmethode. Bei $n=50$ und $n=100$ liefern sowohl Trapez- als auch Simpsonmethode Ergebnisse im Bereich der erlaubten Abweichung. Die Rechteckmethode fällt, genauso wie bei der Polynomfunktion, in allen 3 Tests durch. Dies deutet darauf hin, dass die numerische Integration besonders unzuverlässig bei kleinen Werten von n ist.

3.3 Limitation der Rechteckmethode

Wie die Ergebnisse aus 3.2 gezeigt haben, ist die Rechteckmethode nicht geeignet, falls n klein ist. Je nach Akzeptanzkriterien muss sie auch nicht für größere n geeignet sein. Des Weiteren zeigt diese Methode ihre Schwäche bei trigonometrischer Funktion zu haben, wo sie das Ergebnis mit selbst $n = 100$ mit 15,761% Abweichung approximiert. Sie erweist sich daher als ungeeignet für die numerische Integration, da die Trapez- und Simpsonmethode zuverlässigere Ergebnisse, mit gleichem Aufwand liefern.

3.4 Limitation der Simpsonmethode

Die Simpsonmethode funktioniert nur für gerade Anzahl n. Ist die Anzahl n ungerade, so muss eine andere Formel angewendet werden, die sog. Pulcherrima [1, S. 9]. Ansonsten liefert die Simpsonmethode bei ungeraden Werten n ungenaue Ergebnisse und behandelt das ungerade n so, als wäre n gerade, da z.B.: für $n=27$ und der ersten Summationsgrenze

$\frac{n}{2} = \frac{27}{2} = 13{,}5$ die Summationsformel nur bis 13 rechnet. Somit wurde nur für $n=26$ gerechnet. Allerdings wurde die Pulcherrima-Methode bei der Implementierung nicht beachtet. In einer separaten Testumgebung wurde die Pulcherrima Methode getestet. Dies zeigte nur minimale Verbesserung und die Ergebnisse der Tests waren minimal unterschiedlich. Aufgrund des geringen Mehrwerts im Verhältnis zur Komplexität und unter Berücksichtigung der Aufgabenstellung wird daher auf die Integration der Pulcherrima-Methode verzichtet. Die Tests und Parameter wurden so gewählt, sodass sie alle Methoden vergleichbar machen.

3.5 Fazit

Die Trapezmethode erweist sich als die beste Methode. Sie bietet einen idealen Kompromiss zwischen Genauigkeit und Anwendungsflexibilität. Da die Rechteckmethode aufgrund ihrer massiven Abweichungen als ungeeignet eingestuft wurde und Simpson bei ungeradem n fehleranfällig und an sich komplex ist, stellt die Trapezmethode die universellste und sicherste Wahl für die Implementierung dar.

4 Glossar der Abkürzungen

CON.c – CON_Controller.c

CON.h – CON_Controller.h

APP.c – APP_Application.c

APP.h – APP_Application.h

NIN.c – NIN_Integration.c

NIN.h – NIN_Integration.h

z.B. – zum Beispiel

d.h. – das heißt

vgl. – vergleiche

sog. - sogenannte

5 Appendix

5.1 Berechnung des Integrals der Polynomfunktion

$$F(x) = \int (4x^4 + 7x^3 - 4x + 14)dx$$

$$F(x) = 4\frac{x^5}{5} + 7\,\frac{x^4}{4} - 4\frac{x^2}{2} + 14x \;\rightarrow 0{,}8x^5 + 1{,}75x^4 - 2x^2 + 14x$$

$$\int_0^{20} f(x)dx = F(20) - F(0)$$

$$F(20) = 0{,}8(20)^5 + 1{,}75(20)^4 - 2(20)^2 + 14(20)$$

$$F(20) = 2\,839\,480$$

$$F(0) = \;0{,}8(0)^5 + 1{,}75(0)^4 - 2(0)^2 + 14(0)$$

$$F(0) = 0$$

$$\int_0^{20} f(x)dx = 2\,839\,480 - 0$$

$$\underline{\underline{\int_0^{20} f(x)dx = 2\,839\,480}}$$

5.2 Berechnung des Integrals der trigonometrischen Funktion

$$F(x) = \int 18\sin(x)dx$$

$$F(x) = \;-18\cos(x)$$

$$\int_0^{20} f(x)dx = F(20) - F(0)$$

$$F(20) = \;-18\cos(20)$$

$$F(20) \;\approx\; -7{,}345477$$

$$F(0) = \;-18\cos\,(0)$$

$$F(0) = \;-18$$

$$\int_0^{20} f(x)dx = \;-7{,}345477 - (-18)$$

$$\underline{\underline{\int_0^{20} f(x)dx \;\approx 10,655}} \qquad \text{(aufgerundet auf 3 Dezimalstellen)}$$

6 Nutzung von KI

6.1 Prompt 1

„Ich möchte die Variablen a, b, n, h, der Numerischen Integrationsverfahren (Rechteck-, Trapez- und Simpsonmethode) kurz vorstellen, was sie bedeuten, da sie in den 3 Methoden gleiche Bedeutung haben. Schreibe mir kurze Beschreibung in Fließtext“

6.2 Prompt 2

"Die polynomische Funktion soll die Methoden testen, falls das Integral schnell steigt und die Fläche unter der Kurve sehr groß wird. Die trigonometrische Funktion hingegen soll die Methoden testen, wie sie sich verhalten, falls der Graph der Funktion unterhalb der x-Achse verläuft und wie negative Integrale die Ergebnisse beeinflussen."
Formuliere den Text etwas professioneller

7 Literaturverzeichnis

[1] U. Telle, „Proseminar 1092 - Analysis Numerische Integration“, Dez. 2002.